



HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

LẬP TRÌNH C CƠ BẢN



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

LẬP TRÌNH C CƠ BẢN

CÂY – PHẦN 1

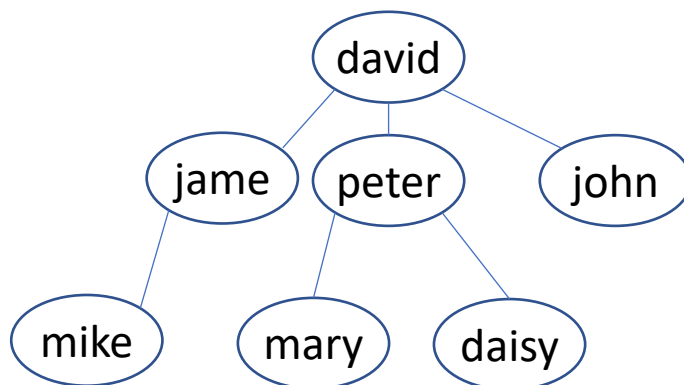
ONE LOVE. ONE FUTURE.

Thao tác với cây tổng quát

- Mỗi nút của cây có cấu trúc dữ liệu như sau

```
typedef struct Node{  
    char name[256];  
    struct Node* leftMostChild; // pointer to the left-most child  
    struct Node* rightSibling; // pointer to the right sibling  
}Node;
```

- Dữ liệu về cây được lưu ở file text với định dạng như sau:
 - Mỗi dòng lưu 1 dãy các xâu ký tự $s_0, s_1, \dots, s_k$ kết thúc bởi $\$$ trong đó $s_1, s_2, \dots, s_k$ là con của s_0 từ trái qua phải (s_1 là con trái nhất của s_0) (ghi chú: mỗi dòng (trừ dòng 1), s_0 là con của một nút nào đó đã xuất hiện trong dòng trước đó).
 - File được kết thúc bởi $$$$



```
david jame peter john $  
peter mary daisy $  
jame mike $  
$$
```

- **Lab 01.** Viết chương trình C chạy ở chế độ tương tác để thao tác với cây biểu diễn sơ đồ phả hệ gia đình với các lệnh sau:
 - Load <filename>: nạp dữ liệu từ <filename> vào bộ nhớ và dựng cây
 - FindChildren <name>: in ra màn hình danh sách con của <name>
 - AddChild <name> <child>: thêm một con <child> vào cuối danh sách con của <name>
 - Print: In ra màn hình danh sách tất cả các thành viên trong gia đình
 - Height <name>: in ra chiều cao của <name> trên cây
 - Count: đếm số thành viên của gia đình (số nút của cây)
 - Store <filename>: Lưu dữ liệu cây ra file <filename>

Cây gia phả

```
#include <stdio.h>
// Khai báo cấu trúc cây
typedef struct Node{
    char name[256];
    struct Node* leftMostChild;
    struct Node* rightSibling;
}Node;
Node* root;

// Tạo một node mới
Node* makeNode(char* name){
    Node* p = (Node*)malloc(sizeof(Node));
    strcpy(p->name,name);
    p->leftMostChild = NULL; p->rightSibling = NULL;
    return p;
}
```

Cây gia phả

```
Node* find(Node* r, char* name){
    if(r == NULL) return NULL;
    if(strcmp(r->name,name) == 0) return r;
    Node* p = r->leftMostChild;
    while(p != NULL){
        Node* q = find(p,name);
        if(q != NULL) return q;
        p = p->rightSibling;
    }
}
```

Cây gia phả

```
Node* addLast(Node* p, char*name){
    if(p == NULL) return makeNode(name);
    p->rightSibling = addLast(p->rightSibling, name);
    return p;
}

void addChild(char*name, char* child){
    Node* r = find(root,name);
    if(r == NULL) return;
    r->leftMostChild = addLast(r->leftMostChild,child);
}
```

Cây gia phả

```
void printTree(Node* r){
    if(r == NULL) return;
    printf("%s: ", r->name);
    Node* p = r->leftMostChild;
    while(p != NULL){
        printf("%s ", p->name);
        p = p->rightSibling;
    }
    printf("\n");
    p = r->leftMostChild;
    while(p != NULL){
        printTree(p);
        p = p->rightSibling;
    }
}
```

Cây gia phả

```
void printTreeF(Node* r, FILE* f){
    if(r == NULL) return;
    fprintf(f,"%s ",r->name);
    Node* p = r->leftMostChild;
    while(p != NULL){
        fprintf(f,"%s ",p->name);
        p = p->rightSibling;
    }
    fprintf(f," $\n");
    p = r->leftMostChild;
    while(p != NULL){
        printTreeF(p,f);
        p = p->rightSibling;
    }
}
```

Cây gia phả

```
void processFind(){
    char name[256];    scanf("%s",name);
    Node* p = find(root,name);
    if(p == NULL) printf("Not Found %s\n",name);
    else printf("Found %s\n",name);
}
```

Cây gia phả

```
void processFindChildren(){
    char name[256];    scanf("%s",name);
    Node* p = find(root,name);
    if(p == NULL) printf("Not Found %s\n",name);
    else{
        printf("Found %s with children: ",name);
        Node* q = p->leftMostChild;
        while(q != NULL){
            printf("%s ",q->name);    q = q->rightSibling;
        }
    }
    printf("\n");
}
```

Cây gia phả

```
int height(Node* p){
    if(p == NULL) return 0;
    int maxH = 0;
    Node* q = p->leftMostChild;
    while(q != NULL){
        int h = height(q);
        maxH = maxH < h ? h : maxH;
        q = q->rightSibling;
    }
    return maxH + 1;
}
```

Cây gia phả

```
void processHeight(){
    char name[256];
    scanf("%s",name);
    Node* p = find(root,name);
    if(p == NULL) printf("Not Found %s\n",name);
    else{
        printf("Found %s having height = %d\n",name,height(p));
    }
}
```

Cây gia phả

```
int count(Node* r){
    if(r == NULL) return 0;
    int cnt = 1;
    Node* q = r->leftMostChild;
    while(q != NULL){
        cnt += count(q);
        q = q->rightSibling;
    }
    return cnt;
}

void processCount(){
    printf("Number of members is %d\n",count(root));
}
```

Cây gia phả

```
void processStore(){
    char filename[256];
    scanf("%s",filename);
    FILE* f = fopen(filename,"w");
    printTreeF(root,f);
    fprintf(f,"$$");
    fclose(f);
}
```

Cây gia phả

```
void freeTree(Node* r){
    if(r == NULL) return;
    Node* p = r->leftMostChild;
    while(p != NULL){
        Node* sp = p->rightSibling;
        freeTree(p);
        p = sp;
    }
    printf("free node %s\n",r->name); free(r);
    r = NULL;
}
```

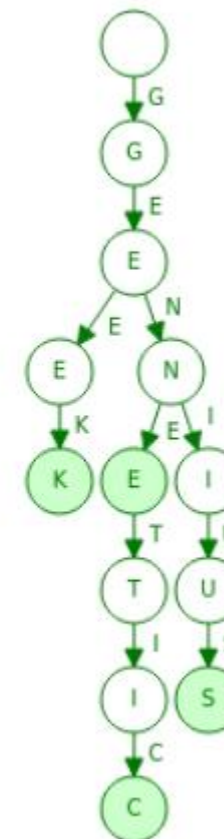
Cây gia phả

```
void main(){
    while(1){
        char cmd[256];
        printf("Enter command: "); scanf("%s",cmd);
        if(strcmp(cmd,"Quit") == 0) break;
        else if(strcmp(cmd,"Load")==0) processLoad();
        else if(strcmp(cmd,"Print")==0) processPrint();
        else if(strcmp(cmd,"Find")==0) processFind();
        else if(strcmp(cmd,"FindChildren")==0) processFindChildren();
        else if(strcmp(cmd,"Height")==0) processHeight();
        else if(strcmp(cmd,"Count")==0) processCount();
        else if(strcmp(cmd,"AddChild")==0) processAddChild();
        else if(strcmp(cmd,"Store")==0) processStore();
    }
    freeTree(root);
}
```

- Yêu cầu thêm
 - Viết hàm xóa 1 nút trên cây
 - Viết hàm xóa toàn bộ cây (giải phóng bộ nhớ cấp phát động)

Cây tiền tố

- **Lab02.** Cây tiền tố (Prefix Tree), còn được gọi là Trie (được phát âm là "try"),
 - là một cấu trúc dữ liệu cây đặc biệt được sử dụng để lưu trữ và tra cứu các chuỗi ký tự có thể chia thành các từ hoặc tiền tố (prefix).
 - Cây tiền tố thường được sử dụng để tìm kiếm, tìm kiếm từ điển, và kiểm tra tính tồn tại của các chuỗi.



“geek”, “genius”, “gene” và “genetic”,

- Hãy xây dựng chương trình
 - Đọc vào danh sách các từ (chỉ gồm chữ cái tiếng anh, viết thường) và xây dựng cây prefix
 - Nhập vào 1 từ, kiểm tra từ đó có trong danh sách các từ ban đầu hay không
 - Nhập vào 1 cụm prefix, in ra các từ có cùng bắt đầu bởi cụm đó trong cây vừa dựng
 - Xóa 1 từ trên cây prefix vừa dựng
 - Giải phóng bộ nhớ cấp phát cho cây

Cây tiền tố

```
/*Minh họa các thao tác cơ bản trên prefix tree*/
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#define ALPHABET_SIZE 26

// Định nghĩa cấu trúc dữ liệu nút cho cây prefix
// cây chỉ chứa các từ tiếng anh chữ thường
// Dùng mảng số 0-25 cho chữ cái từ a-z (trừ độ lệch -
'a')
struct TrieNode {
    struct TrieNode* children[ALPHABET_SIZE];
    bool isEndOfWord; // đánh dấu trạng thái kết thúc
    của từ
};
```

```
// Hàm tạo nút mới bằng cách cấp phát động nút mới
// và khởi tạo giá trị các nút con của nút mới là
NULL
struct TrieNode* createNode() {
    struct TrieNode* pNode = (struct
TrieNode*)malloc(sizeof(struct TrieNode));
    pNode->isEndOfWord = false;
    for (int i = 0; i < ALPHABET_SIZE; i++) {
        pNode->children[i] = NULL;
    }
    return pNode;
}
```

```
// Hàm tạo nút mới bằng cách cấp phát động nút mới
// và khởi tạo giá trị các nút con của nút mới là NULL
struct TrieNode* createNode() {
    struct TrieNode* pNode = (struct
TrieNode*)malloc(sizeof(struct TrieNode));
    pNode->isEndOfWord = false;
    for (int i = 0; i < ALPHABET_SIZE; i++) {
        pNode->children[i] = NULL;
    }
    return pNode;
}
```

```
// Hàm thêm từ vào cây prefix
void insertWord(struct TrieNode* root, const char*
word) {
    struct TrieNode* currentNode = root;
    for (int i = 0; word[i] != '\0'; i++) {
        int index = word[i] - 'a';
        if (!currentNode->children[index]) {
            currentNode->children[index] =
createNode();
        }
        currentNode = currentNode->children[index];
    }
    currentNode->isEndOfWord = true;
}
```

Cây tiền tố

```
// Hàm kiểm tra từ có trong cây prefix hay không
bool searchWord(struct TrieNode* root, const char* word)
{
    struct TrieNode* currentNode = root;
    for (int i = 0; word[i] != '\0'; i++) {
        int index = word[i] - 'a';
        if (!currentNode->children[index]) {
            return false;
        }
        currentNode = currentNode->children[index];
    }
    return (currentNode != NULL && currentNode->isEndOfWord);
}
```

// Hàm duyệt và in ra tất cả các từ trong cây prefix
word là mảng chứa các ký tự ở phía trước. Level là số lượng ký tự trong mảng word tới thời điểm hiện tại

```
void traverseTrie(struct TrieNode* root, char word[],
int level) {
    if (root->isEndOfWord) {
        word[level] = '\0';
        printf("%s\n", word);
    }
    for (int i = 0; i < ALPHABET_SIZE; i++) {
        if (root->children[i]) {
            word[level] = 'a' + i;
            traverseTrie(root->children[i], word,
level + 1);
        }
    }
}
```

Thao tác với cây tổng quát

```
// Hàm kiểm tra từ có trong cây prefix hay không
void printPrefixWords(struct TrieNode* root, const char* word) {
    struct TrieNode* currentNode = root;
    // đi theo danh sách prefix
    for (int i = 0; word[i] != '\0'; i++) {
        int index = word[i] - 'a';
        if (!currentNode->children[index]) {
            return ;
        }
        currentNode = currentNode->children[index];
    }
    if(currentNode == NULL) return;

    // Duyệt và in ra tất cả các từ trong cây prefix
    char prefixWord[50];
    strcpy(prefixWord, word);
    int level = strlen(prefixWord);

    traverseTrie(currentNode,prefixWord,level);
}
```

Cây tiền tố

```
// Hàm main để thử nghiệm cây prefix
int main() {
    struct TrieNode* root = createNode();

    // Thêm các từ vào cây prefix
    insertWord(root, "hello");
    insertWord(root, "world");
    insertWord(root, "hi");
    insertWord(root, "hey");

    // Kiểm tra từ có trong cây prefix hay không
    printf("Does 'hello' exist in the trie? %s\n", searchWord(root, "hello") ? "Yes" :
"No");
    printf("Does 'world' exist in the trie? %s\n", searchWord(root, "world") ? "Yes" :
"No");
    printf("Does 'hey' exist in the trie? %s\n", searchWord(root, "hey") ? "Yes" : "No");
    printf("Does 'how' exist in the trie? %s\n", searchWord(root, "how") ? "Yes" : "No");

    // in ra các từ có cùng prefix là "he" --> hello và hey
    printPrefixWords(root, "he");

    return 0;
}
```

- Yêu cầu thêm
 - Sửa lại hàm đọc để có thể đọc vào danh sách các từ lấy từ 1 đoạn văn (chuyển hết các ký tự in hoa về thường)
 - Sửa lại cấu trúc để tìm xem các tổ hợp prefix có k ký tự nào có tần số xuất hiện lớn nhất trên đoạn văn vừa nhập
 - Thêm hàm xóa 1 từ trên cây prefix
 - Hoàn thiện hàm xóa toàn bộ cây (giải phóng bộ nhớ cấp phát động của từng phần tử)

A large graphic on the left side of the slide. It features a dark blue background with a circular pattern of red dots of varying sizes, creating a sense of depth and movement. The word "HUST" is centered within this graphic in a white, bold, sans-serif font.

HUST

THANK YOU !